SnapSense AI — Architecture & Design Document

> **SnapSense AI** is a full-stack SaaS application that uses AI-powered vision models to automatically classify screenshots, organize them into Google Drive, log metadata to Google Sheets, create Calendar events, and manage Google Tasks — all from a single drag-and-drop upload.

Table of Contents

1. High-Level Architecture
2. Technology Stack
3. Project Structure
4. Server Architecture
    - Entry Point & Middleware Pipeline
    - Database Models
    - Authentication System
    - Middleware Layer
    - Routing Layer
    - Service Layer
    - Webhook System
5. Client Architecture
    - App Shell & Routing
    - State Management
    - Pages
    - Components
    - Hooks
    - Styling & Theming
6. Core User Flows
    - Anonymous Upload Flow
    - Authenticated Upload Flow
    - Session Merge Flow
    - Free-to-Pro Upgrade Flow
    - Subscription Lifecycle
    - Screenshot Deletion Flow
7. AI Classification Engine
8. Google Integration Layer
9. Billing & Payments
10. Security Architecture
11. API Reference
12. Free vs Pro Feature Matrix
13. Data Flow Diagrams
14. Environment Variables

---

1. High-Level Architecture

```
┌──────────────────────────────────────────────────────────────────────────────┐
│                                CLIENT (React + Vite)                         │
│  ┌──────────┐  ┌──────────┐  ┌──────────────┐  ┌──────────────────────────┐ │
│  │ Landing  │  │ Dashboard │  │ CategoryView │  │ Shared Components       │ │
│  │ Page     │  │ Page      │  │ Page         │  │ Navbar, UploadZone,     │ │
│  │ (anon)   │  │ (auth)    │  │ (auth)       │  │ ThemeToggle, Modals     │ │
│  └──────────┘  └──────────┘  └──────────────┘  └──────────────────────────┘ │
│       │             │              │                      │                  │
│       └─────────────┴──────────────┴──────────────────────┘                  │
│                     │              │                                         │
│                     ▼              ▼                                         │
│              ┌──────────────────────────────┐                                │
│              │     AuthContext + useBilling  │                               │
│              └──────────────────────────────┘                                │
└──────────────────────────────────────────────────────────────────────────────┘
                                  │
                                  │  HTTP (Axios)
                                  │  Proxy: /auth → :5000, /api → :5000
                                  ▼
┌──────────────────────────────────────────────────────────────────────────────┐
│                          SERVER (Express.js + Node.js)                       │
│                                                                              │
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────────┐ │
│  │ Auth Routes  │  │ Screenshot   │  │ Billing      │  │ Webhook Routes   │ │
│  │ /auth/*      │  │ Routes       │  │ Routes       │  │ /api/webhook/*   │ │
│  │              │  │ /api/ss/*    │  │ /api/bill/*  │  │                  │ │
│  └──────────────┘  └──────────────┘  └──────────────┘  └──────────────────┘ │
│        │                │                │                │                  │
│  ┌─────▼────────────────▼────────────────▼────────────────▼──────────────┐  │
│  │                        MIDDLEWARE LAYER                               │
│
│  │  requireAuth │ checkSubscriptionExpiry │ checkUploadLimit │ requirePro │
│
│  └──────────────────────────────────────────────────────────────────────┘  │
│                                     │                                        │
│  ┌──────────────────────────────────▼──────────────────────────────────┐   │
│  │                          SERVICE LAYER                               │   │
│  │  groqService │ driveService │ sheetsService │ calendarService       │   │
│  │  googleTasks │ lemonSqueezyService                                  │   │
│  └──────────────────────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────────────────────┘
        │          │          │          │          │
        ▼          ▼          ▼          ▼          ▼
   ┌─────────┐ ┌─────────┐ ┌─────────┐ ┌─────────┐ ┌──────────────┐
   │ Groq    │ │ Google  │ │ Google  │ │ Google  │ │ Lemon        │
   │ Vision  │ │ Drive   │ │ Sheets  │ │ Calendar│ │ Squeezy      │
   │ AI API  │ │ API     │ │ API     │ │ + Tasks │ │ Payments API │
   └─────────┘ └─────────┘ └─────────┘ └─────────┘ └──────────────┘
                                 │
                            ┌────▼────┐
                            │ MongoDB │
                            │ Atlas   │
                            └─────────┘
```

2. Technology Stack

Backend

Technology	Version	Purpose
Node.js	—	Runtime
Express.js	4.21.2	Web framework
MongoDB + Mongoose	8.9.5	Database & ODM
Passport.js	0.7.0	Authentication framework
passport-google-oauth20	2.0.0	Google OAuth 2.0 strategy
express-session	1.18.1	Session management
connect-mongo	5.1.0	MongoDB session store
googleapis	144.0.0	Google Drive, Sheets, Calendar, Tasks
groq-sdk	0.12.0	Groq Vision AI for classification
@lemonsqueezy/lemonsqueezy.js	3.0.0	Payment processing SDK
jsonwebtoken	9.0.2	JWT utilities
multer	1.4.5	Multipart file upload handling
cors	2.8.5	Cross-origin resource sharing
cookie-parser	1.4.7	Cookie parsing
dotenv	16.4.7	Environment variable management
nodemon	3.1.14	Dev auto-restart

Frontend

Technology	Version	Purpose
React	19.2.0	UI framework
React DOM	19.2.0	DOM rendering
React Router DOM	7.13.1	Client-side routing
Vite	—	Build tool & dev server
Axios	1.13.6	HTTP client
Framer Motion	12.35.0	Animations & transitions
Lucide React	0.577.0	Icon library

Technology	Version	Purpose
react-dropzone	15.0.0	Drag-and-drop file uploads
react-hot-toast	2.6.0	Toast notifications

External APIs

Service	Purpose
Groq Vision AI (`meta-llama/llama-4-scout-17b-16e-instruct`)	Screenshot classification
Google Drive API	File storage & organization
Google Sheets API	Metadata logging (4 tabs)
Google Calendar API	Event creation for time-sensitive items
Google Tasks API	Task creation for actionable items
Lemon Squeezy	Subscription billing & payments

Infrastructure

Component	Technology
Database	MongoDB Atlas
Session Store	MongoDB (connect-mongo)
Hosting	Node.js (serves React build in production)
Font CDN	Google Fonts (Inter, Space Grotesk)

3. Project Structure

```
SnapSense/
├── package.json                    # Root workspace scripts
├── README.md                       # Project documentation
│
├── client/                         # React frontend (Vite)
│   ├── package.json                # Frontend dependencies
│   ├── index.html                  # HTML entry point (Inter + Space Grotesk
fonts)
│   ├── vite.config.js              # Vite config: port 5173, proxy to :5000
│   ├── eslint.config.js            # ESLint: react-hooks, react-refresh
│   ├── public/                     # Static assets
│   └── src/
│       ├── main.jsx                # React root: BrowserRouter → AuthProvider
→ App + Toaster
```

```
│       ├── App.jsx                 # Route definitions + ProtectedRoute
wrapper
│       ├── App.css                 # Minimal root styles
│       ├── index.css               # Full design system: colors, tokens,
themes
│       ├── assets/                 # Static assets (images, etc.)
│       ├── config/                 # (Empty — reserved for client config)
│       ├── context/
│       │   └── AuthContext.jsx     # Auth state provider: user, login, logout,
fetchUser
│       ├── hooks/
│       │   └── useBilling.js       # Billing hook: checkout, polling, Lemon
Squeezy embed
│       ├── pages/
│       │   ├── Landing.jsx         # Public landing page with anonymous upload
│       │   ├── Dashboard.jsx       # Authenticated dashboard: stats, upload,
recent
│       │   └── CategoryView.jsx    # Category gallery with delete support
│       └── components/
│           ├── Navbar.jsx          # Fixed top nav: logo, user menu, PRO badge
│           ├── ThemeToggle.jsx     # Light/dark theme toggle (localStorage)
│           ├── Stardust.jsx        # Canvas particle background animation
│           ├── UploadZone.jsx      # Drag-drop multi-file upload with progress
│           ├── SubscriptionBanner.jsx  # PRO status banner with days remaining
│           ├── UpgradeModal.jsx    # Plan selection modal with Lemon Squeezy
checkout
│           ├── PricingPlans.example.jsx        # Reference: pricing page
template
│           └── SubscriptionStatus.example.jsx  # Reference: subscription
widget template
│
└── server/                         # Express.js backend
    ├── package.json                # Server dependencies
    └── src/
        ├── index.js                # Server entry: Express setup, MongoDB,
routes
        ├── config/
        │   ├── passport.js         # Google OAuth strategy +
serialize/deserialize
        │   └── lemonsqueezy.js     # Lemon Squeezy SDK init + checkout +
cancel
        ├── middleware/
        │   ├── auth.js             # requireAuth: session-based auth guard
        │   └── tierCheck.js        # checkSubscriptionExpiry, requirePro,
checkUploadLimit
        ├── models/
        │   ├── User.js             # User schema: Google profile, tokens,
tier, subscription
        │   ├── Screenshot.js       # Classified screenshot: category,
metadata, Drive/Calendar links
        │   ├── Action.js           # Anonymous session pending action (24hr
TTL)
        │   ├── AnonymousSession.js # Anonymous session tracker (24hr TTL)
```

```
        │   └── Payment.js          # Payment ledger with webhook event log
        ├── routes/
        │   ├── auth.js             # OAuth flow + logout + /me + /merge
        │   ├── screenshots.js      # Upload, list, stats, category filter,
delete
        │   ├── billing.js          # Checkout, status, pricing, history,
subscription
        │   └── webhook.js          # Lemon Squeezy webhook handler (9 event
types)
        └── services/
            ├── groqService.js      # AI classification via Groq Vision (Llama
4 Scout)
            ├── driveService.js     # Google Drive: folders, upload, delete
            ├── sheetsService.js    # Google Sheets: 4-tab spreadsheet, append,
delete rows
            ├── calendarService.js  # Google Calendar: event create/delete
            ├── googleTasks.js      # Google Tasks: task create
            └── lemonSqueezyService.js  # Lemon Squeezy: checkout,
subscription, cancel
```

---

4. Server Architecture

4.1. Entry Point & Middleware Pipeline

File: `server/src/index.js`

The server boots in the following order:

```
1. Load environment variables (dotenv)
2. Create Express app
3. Configure CORS (CLIENT_URL origin, credentials: true)
4. Mount raw body parser for webhooks (BEFORE express.json)
5. Mount express.json (50MB limit) + express.urlencoded + cookieParser
6. Health check endpoint: GET /health → { status, dbState }
7. Connect to MongoDB Atlas (with retry logic on failure)
8. Configure session store (MongoDB, 7-day maxAge, httpOnly cookie)
9. Initialize Passport (session-based authentication)
10. Mount global middleware: checkSubscriptionExpiry (on every auth request)
11. Mount route groups:
    - /auth          → authRoutes
    - /api/screenshots → screenshotRoutes
    - /api/billing   → billingRoutes
    - /api/webhook   → webhookRoutes
12. Production: serve client/dist as static + SPA fallback
13. Listen on PORT (default 5000)
```

Key Design Decisions:

- Raw body parser is mounted **before** `express.json()` so that webhook signature verification receives the raw request body for HMAC-SHA256 validation.
- JSON body limit is set to 50MB to accommodate base64-encoded image payloads.
- MongoDB connection uses retry logic with exponential backoff.
- Session cookie is `httpOnly` and `secure` in production (SameSite: lax).
- `checkSubscriptionExpiry` runs globally as middleware to auto-downgrade expired PRO users on every authenticated request.

4.2. Database Models

4.2.1. User Model

File: `server/src/models/User.js`

The central user identity. Created on first Google OAuth login.

Field	Type	Description
`googleId`	String (unique, required)	Google OAuth subject identifier
`email`	String (required)	User email from Google profile
`name`	String (required)	Display name from Google profile
`picture`	String (default: `''`)	Google profile avatar URL
`accessToken`	String	Google OAuth access token (refreshed on re-login)
`refreshToken`	String	Google OAuth refresh token (long-lived)
`driveRootFolderId`	String	ID of "📷 SnapSense AI" root folder in Drive
`driveCategoryFolders`	Map<String, String>	Map of category name → Drive folder ID
`sheetsId`	String	ID of the user's SnapSense Sheets spreadsheet
`tier`	Enum: `'free'` | `'pro'` (default: `'free'`)	Current subscription tier
`screenshotCount`	Number (default: 0)	Screenshots uploaded in current billing cycle

Field	Type	Description
`countResetAt`	Date	When the monthly screenshot counter resets (30 days from first upload)
`totalUploads`	Number	Lifetime total uploads
`subscription.lsOrderId`	String	Lemon Squeezy order ID
`subscription.lsSubscriptionId`	String	Lemon Squeezy subscription ID
`subscription.status`	Enum: `'none'` | `'active'` | `'payment_failed'` | `'cancelled'` | `'expired'` | `'refunded'`	Subscription status
`subscription.currentPeriodEnd`	Date	When the current billing period expires
`subscription.cancelledAt`	Date	When the user cancelled (grace period until `currentPeriodEnd`)
`createdAt` / `updatedAt`	Date	Mongoose timestamps

Key Behaviors:

- `driveCategoryFolders` is a Mongoose Map — one folder per category, lazily created on first upload to that category.
- `screenshotCount` resets to 0 automatically when `countResetAt` expires (checked in `checkUploadLimit` middleware).
- `subscription.status` transitions are driven entirely by Lemon Squeezy webhooks.

4.2.2. Screenshot Model

File: `server/src/models/Screenshot.js`

Represents a classified, permanently stored screenshot for an authenticated user.

Field	Type	Description
`userId`	ObjectId → User (required)	Owner reference
`originalName`	String (default: `'screenshot.png'`)	Original filename
`mimeType`	String (default: `'image/png'`)	MIME type
`fileHash`	String (required)	MD5 hash for duplicate detection

Field	Type	Description
`category`	Enum (14 values, required)	AI-assigned category
`metadata.summary`	String	AI-generated summary of the screenshot
`metadata.date`	String	Detected date (ISO 8601 or null)
`metadata.suggestedAction`	Enum: `'calendar'` | `'task'` | `'sheet'` | `'contact'` | `'none'`	AI-recommended downstream action
`metadata.confidence`	Number (0–1)	Classification confidence score
`metadata.locationCategory`	String	Sub-category for Location screenshots
`metadata.locationName`	String	Detected location name
`metadata.mapLink`	String	Google Maps link for locations
`metadata.quoteAuthor`	String	Author for Quote screenshots
`metadata.quoteGenre`	String	Genre/topic for Quote screenshots
`metadata.transactionType`	Enum: `'Credit'` | `'Debit'` | null	For Payment screenshots
`metadata.transactionTime`	String	Detected transaction time
`metadata.transactionAmount`	String	Detected amount
`metadata.rawAI`	String	Full raw Groq response text
`driveFileId`	String	Google Drive file ID
`driveViewLink`	String	Public Drive view link
`driveThumbnailLink`	String	Drive thumbnail URL
`calendarEventId`	String	Google Calendar event ID
`calendarEventLink`	String	Calendar event URL
`taskId`	String	Google Tasks task ID
`taskLink`	String	Google Tasks URL

Field	Type	Description
`sheetsRowNumber`	Number	Row number in the Sheets log

14 Supported Categories:

1. Location
2. Ticket
3. Wallpaper
4. LinkedIn Profile
5. LinkedIn Post
6. Social Media Post
7. Payment
8. Sensitive Document
9. Contact
10. Mail
11. Quote
12. WhatsApp Chat
13. Study Notes
14. Other

4.2.3. Action Model

File: `server/src/models/Action.js`

Temporary document for anonymous (pre-login) uploads. Designed for merge into the Screenshot collection after authentication.

Field	Type	Description
`sessionId`	String (indexed, required)	Links to `AnonymousSession.sessionId`
`originalName`	String	Original filename
`mimeType`	String	MIME type
`imageBuffer`	Buffer	Raw image bytes stored in MongoDB (temporary)
`fileHash`	String (required)	MD5 hash for dedup
`category`	String (required)	AI-classified category
`metadata`	Object	Same structure as Screenshot metadata
`status`	Enum: `'pending'` | `'completed'` | `'failed'`	Merge status

Field	Type	Description
`screenshotId`	ObjectId → Screenshot	Reference to merged Screenshot (after merge)

TTL: Documents auto-delete after **24 hours** via MongoDB TTL index.

4.2.4. AnonymousSession Model

File: `server/src/models/AnonymousSession.js`

Tracks anonymous upload sessions (identified by a UUID stored in the client's localStorage).

Field	Type	Description
`sessionId`	String (unique, required)	UUID generated client-side
`createdAt`	Date	Session creation time

TTL: Auto-deletes after **24 hours**.

4.2.5. Payment Model

File: `server/src/models/Payment.js`

Immutable ledger record for every payment event.

Field	Type	Description
`userId`	ObjectId → User (indexed, required)	Payer
`lsOrderId`	String (unique, indexed, required)	Lemon Squeezy order ID
`lsSubscriptionId`	String	Linked subscription ID
`amount`	Number (required)	Amount in paise (₹ × 100)
`currency`	String (default: `'INR'`)	Currency code
`status`	Enum: `'pending'` | `'paid'` | `'failed'` | `'refunded'`	Payment status
`webhookEvents`	Array of `{ eventName, receivedAt, lsEventId, payload }`	Audit log of all webhook hits
`idempotencyKeys`	Array of String	Prevents duplicate processing

4.3. Authentication System

Files: `server/src/config/passport.js`, `server/src/routes/auth.js`

```
┌──────────┐      GET /auth/google      ┌──────────────────┐
│  Client  │ ──────────────────────────► │  Passport OAuth  │
│  Browser │                             │  Redirect to     │
│          │ ◄────────────────────────── │  Google Consent  │
│          │     302 → Google            └──────────────────┘
│          │                                      │
│          │     GET /auth/google/callback        │ Google returns code
│          │ ◄────────────────────────────────────┘
│          │                             ┌──────────────────┐
│          │ ──────────────────────────► │ Passport verify  │
│          │   with ?code=xxx            │ callback:        │
│          │                             │ 1. Exchange code │
│          │                             │ 2. Find/Create   │
│          │                             │    User in DB    │
│          │                             │ 3. Store tokens  │
│          │   302 → /dashboard          │ 4. Serialize to  │
│          │ ◄────────────────────────── │    session       │
└──────────┘                             └──────────────────┘
```

OAuth Scopes Requested:

Scope	Purpose
`profile`	User name and avatar
`email`	User email address
`drive.file`	Create/manage files in Drive
`spreadsheets`	Read/write Google Sheets
`calendar.events`	Create/delete Calendar events
`tasks`	Create Google Tasks

Session Configuration:

- Store: MongoDB via `connect-mongo`
- Cookie name: `connect.sid`
- Max age: 7 days
- Flags: `httpOnly: true`, `secure: production`, `sameSite: 'lax'`

Serialization: Only `user._id` is stored in the session. On each request, `deserializeUser` fetches the full user object from MongoDB.

Token Handling: On every re-login, `accessToken` and `refreshToken` are updated on the User document. The `accessType: 'offline'` and `prompt: 'consent'` flags ensure a refresh token is always

issued.

4.4. Middleware Layer

4.4.1. `requireAuth`

File: `server/src/middleware/auth.js`

```
// Guards all authenticated endpoints
// Checks: req.isAuthenticated() && req.user
// Returns: 401 { error: 'Authentication required' }
```

Simple session-based gate. Applied to all endpoints that require a logged-in user.

4.4.2. `checkSubscriptionExpiry`

File: `server/src/middleware/tierCheck.js`

Runs **globally** on every authenticated request. Silently auto-downgrades users whose subscription has expired:

```
if user.subscription.currentPeriodEnd < Date.now()
   AND user.tier === 'pro'
   AND user.subscription.status === 'active'
then:
   user.tier = 'free'
   user.subscription.status = 'expired'
   user.save()
```

This ensures users never access PRO features past their subscription's expiry, even if the Lemon Squeezy webhook is delayed.

4.4.3. `checkUploadLimit`

File: `server/src/middleware/tierCheck.js`

Guards the authenticated upload endpoint.

```
PRO tier → always pass (unlimited)

FREE tier:
  - If countResetAt has passed (or is null):
      → Reset screenshotCount to 0
```

```
        → Set countResetAt to now + 30 days
    - If screenshotCount >= 10:
        → Return 402 { error: 'LIMIT_REACHED', limit: 10, current: N, resetAt }
    - Otherwise → pass
```

4.4.4. `requirePro`

File: `server/src/middleware/tierCheck.js`

Feature gate for PRO-only endpoints.

```
if user.tier !== 'pro' → 403 { error: 'PRO_REQUIRED', feature, message }
```

4.5. Routing Layer

4.5.1. Auth Routes (`/auth/*`)

File: `server/src/routes/auth.js`

Method	Path	Auth	Middleware	Description
GET	`/auth/google`	No	Passport OAuth	Initiates Google OAuth flow. Redirects to Google consent screen.
GET	`/auth/google/callback`	Passport	—	OAuth callback. On success → redirect `/dashboard`. On fail → redirect `/?error=auth_failed`.
POST	`/auth/logout`	Yes	—	Destroys session, clears `connect.sid` cookie.
GET	`/auth/me`	Yes	—	Returns current user (excluding sensitive token fields).
POST	`/auth/merge`	Yes	requireAuth	Merges anonymous session Actions into authenticated user's account.

Merge Endpoint Detail (`POST /auth/merge`):

Request body: `{ sessionId, verifiedCards: [{ actionId, category, metadata }] }`

Process:

1. Find all pending Actions for `sessionId`
2. Atomically claim each Action (set `status = 'completed'`, prevents double-processing)

3. Cross-reference with `verifiedCards` (client-submitted confirmed classifications)
4. For each Action:
    - Upload `imageBuffer` to Google Drive (into category folder)
    - Create Google Calendar event (if applicable)
    - Create Google Task (if applicable)
    - Log to Google Sheets (if applicable)
    - Create permanent `Screenshot` document
    - Increment `screenshotCount` and `totalUploads`
5. Delete the `AnonymousSession`
6. Return `{ merged: N, results: [...] }`

Free-tier limit check: If merge would push count over 10, returns 402 **without** deleting the session (user can upgrade and retry).

4.5.2. Screenshot Routes (`/api/screenshots/*`)

File: `server/src/routes/screenshots.js`

Method	Path	Auth	Middleware	Description
POST	`/api/screenshots/upload`	Yes	requireAuth, checkUploadLimit	Upload and classify a screenshot (authenticated)
POST	`/api/screenshots/upload/anonymous`	No	—	Upload and classify without auth (max 3 per session)
GET	`/api/screenshots`	Yes	requireAuth	Paginated list of user's screenshots
GET	`/api/screenshots/category/:cat`	Yes	requireAuth	All screenshots for a specific category
GET	`/api/screenshots/stats`	Yes	requireAuth	Aggregate counts by category
DELETE	`/api/screenshots/:id`	Yes	requireAuth	Delete screenshot + cascade cleanup

Authenticated Upload Pipeline (`POST /api/screenshots/upload`):

```
1. Multer parses multipart form (in-memory, max 20MB, images only)
2. Generate MD5 hash of file buffer
3. Check duplicate (same hash + userId → 409 Conflict)
4. Classify via Groq AI → { category, metadata }
5. Upload to Google Drive (ensures category folder exists)
6. Conditionally create Calendar event (Tickets, Payments, or
suggestedAction='calendar')
7. Conditionally create Google Task (Study Notes, or suggestedAction='task')
8. Conditionally log to Sheets (Payments → Transactions tab, Quotes → Quotes
tab,
   Locations → Locations tab, or suggestedAction='sheet' → Screenshots tab)
9. Save Screenshot document to MongoDB
10. Increment screenshotCount + totalUploads
11. Return 201 { success, message, screenshot }
```

Anonymous Upload Pipeline (`POST /api/screenshots/upload/anonymous`):

```
1. Read x-session-id from request header
2. Count existing Actions for this session → max 3
3. Create AnonymousSession if not exists
4. Multer parse file
5. Generate MD5 hash
6. Classify via Groq AI
7. Save Action document (with imageBuffer, 24hr TTL)
8. Return { success, actionCard: { actionId, category, metadata, preview } }
```

Delete Pipeline (`DELETE /api/screenshots/:id`):

```
1. Find screenshot by _id + userId (ownership check)
2. If driveFileId → delete from Google Drive
3. If calendarEventId → delete from Google Calendar
4. If sheetsRowNumber → delete row from Sheets
5. Delete Screenshot from MongoDB
6. Decrement screenshotCount and totalUploads
7. Return { success, message }
```

4.5.3. Billing Routes (`/api/billing/*`)

File: `server/src/routes/billing.js`

Method	Path	Auth	Description
POST	`/api/billing/checkout`	Yes	Create Lemon Squeezy checkout session
GET	`/api/billing/status`	Yes	Current tier, counts, limits

Method	Path	Auth	Description
POST	`/api/billing/verify-upgrade`	Yes	Manual upgrade verification (dev)
GET	`/api/billing/pricing`	No	Subscription plans with pricing
GET	`/api/billing/history`	Yes	Last 50 payment records
GET	`/api/billing/subscription`	Yes	Detailed subscription info

Checkout Flow (`POST /api/billing/checkout`):

Request: `{ months: 1 | 3 | 6 | 12 }`

```
1. Validate months ∈ {1, 3, 6, 12}
2. Check no active subscription exists
3. Check no pending payment within 15 minutes (anti-duplicate)
4. Call createCheckoutSession(user, months)
5. Return { checkoutUrl, months }
```

Pricing Structure:

Plan	Price/Month	Total	Discount	Savings
1 month	₹349	₹349	0%	₹0
3 months	₹314	₹942	10%	₹105
6 months	₹297	₹1,781	15%	₹313
12 months	₹279	₹3,350	20%	₹838

4.5.4. Webhook Routes (`/api/webhook/*`)

File: `server/src/routes/webhook.js`

Method	Path	Auth	Description
POST	`/api/webhook/lemonsqueezy`	HMAC-SHA256 signature	Lemon Squeezy event handler
POST	`/api/webhook/test-upgrade`	None (dev only)	Manual test upgrade

4.6. Service Layer

4.6.1. Groq AI Service

File: `server/src/services/groqService.js`

Model: `meta-llama/llama-4-scout-17b-16e-instruct`

Function: `classifyScreenshot(imageBuffer, mimeType)`

```
Input:  Raw image buffer + MIME type
Output: {
  category:          one of 14 categories,
  summary:           natural language description,
  date:              ISO 8601 date or null,
  suggestedAction:   'calendar' | 'task' | 'sheet' | 'contact' | 'none',
  confidence:        0.0 – 1.0,
  quoteAuthor:       string (Quotes only),
  quoteGenre:        string (Quotes only),
  transactionType:   'Credit' | 'Debit' | null (Payments only),
  transactionTime:   string (Payments only),
  transactionAmount: string (Payments only),
  locationName:      string (Locations only),
  locationCategory:  string (Locations only),
  mapLink:           Google Maps URL (Locations only),
  rawAI:             full response text
}
```

Configuration:

- Temperature: 0.1 (near-deterministic for consistent classification)
- Max tokens: 500
- Response format: JSON (parsed from Groq text completion)
- Fallback: Returns `{ category: 'Other' }` on parse failure
- Image encoding: base64 data URL

System Prompt: Contains detailed classification instructions for all 14 categories, including field-specific instructions for each category type (when to extract `quoteAuthor`, when to generate `mapLink`, etc.).

4.6.2. Google Drive Service

File: `server/src/services/driveService.js`

Function	Parameters	Returns	Description
`getDriveClient(user)`	User document	`google.drive` client	Creates OAuth2 client from user tokens
`createFolder(drive, name, parentId)`	Drive client, folder name, parent ID	Folder ID	Creates a Google Drive folder
`ensureUserFolders(user)`	User document	void	Creates root folder "📷 SnapSense AI" + 14 category subfolders

Function	Parameters	Returns	Description
`uploadFileToDrive(user, buffer, filename, mimeType, category)`	User, file data, category	`{ fileId, webViewLink, thumbnailLink }`	Uploads file, sets public read permission
`deleteFileFromDrive(user, fileId)`	User, file ID	void	Deletes file (404 = success)

Folder Structure in Google Drive:

```
📸 SnapSense AI/
├── 📍 Location/
├── 🎟️ Ticket/
├── 🖼️ Wallpaper/
├── 💼 LinkedIn Profile/
├── 📝 LinkedIn Post/
├── 📱 Social Media Post/
├── 💳 Payment/
├── 🔒 Sensitive Document/
├── 👤 Contact/
├── ✉️ Mail/
├── 💬 Quote/
├── 💬 WhatsApp Chat/
├── 📚 Study Notes/
└── 📂 Other/
```

Each file uploaded gets `anyone with link → reader` permission for easy sharing.

4.6.3. Google Sheets Service

File: `server/src/services/sheetsService.js`

Spreadsheet: "📊 SnapSense AI — Screenshot Log"

4 Tabs (Sheets):

Tab 1: Screenshots

Timestamp	Category	Summary	Date Detected	Action Taken	Drive Link	Calendar Event	Confidence

Tab 2: Quotes

Timestamp	Quote Summary	Author	Genre	Drive Link	Confidence

Tab 3: Transactions

Timestamp	Type (Credit/Debit)	Date	Time	Amount	Drive Link	Confidence

Tab 4: Locations

Timestamp	Location Name	Category	Summary	Maps Link	Drive Link	Confidence

Function	Description
`getSheetsClient(user)`	Creates OAuth2 client for Sheets
`ensureUserSheet(user)`	Creates spreadsheet + 4 tabs + headers (idempotent)
`appendRow(user, rowData)`	Appends row to Screenshots tab → returns row number
`appendQuoteRow(user, rowData)`	Appends to Quotes tab
`appendTransactionRow(user, rowData)`	Appends to Transactions tab
`appendLocationRow(user, rowData)`	Appends to Locations tab
`deleteSheetRow(user, rowNumber, sheetName)`	Deletes row by shifting up

Headers are formatted with a blue background on creation.

4.6.4. Google Calendar Service

File: `server/src/services/calendarService.js`

Function	Description
`getCalendarClient(user)`	Creates OAuth2 client for Calendar
`shouldCreateCalendarEvent(category, suggestedAction)`	Returns `true` if category ∈ {Ticket, Payment} OR suggestedAction = 'calendar'
`createCalendarEvent(user, eventData)`	Creates event with reminders. Returns `{ eventId, eventLink }`
`deleteCalendarEvent(user, eventId)`	Deletes event (404/410 = success)

Event Details:

- Summary: `"📸 [category]: [summary]"`
- Time zone: `Asia/Kolkata`
- Default date: 7 days from now (if AI date parse fails)
- Reminders: Pop-up at 60 minutes + Email at 24 hours
- Color: Sage green (`colorId: 3`)

4.6.5. Google Tasks Service

File: `server/src/services/googleTasks.js`

Function	Description
`getTasksClient(user)`	Creates OAuth2 client for Tasks
`shouldCreateGoogleTask(category, suggestedAction)`	Returns `true` if category = 'Study Notes' OR suggestedAction = 'task'
`createGoogleTask(user, taskData)`	Creates task in @default tasklist. Returns `{ taskId, taskLink }`

Task Details:

- Title: `"📷 [summary]"`
- Notes: Includes category, summary, and Drive link
- Task link: Always `https://mail.google.com/tasks/canvas`
- Due date: Parsed from AI metadata (if available)

4.6.6. Lemon Squeezy Service

File: `server/src/services/lemonSqueezyService.js` + `server/src/config/lemonsqueezy.js`

Function	Description
`initializeLemonSqueezy()`	Initializes SDK with API key
`createCheckoutSession(user, months)`	Creates checkout URL with pricing, discounts, and custom metadata
`getSubscriptionDetails(subscriptionId)`	Fetches subscription attributes from Lemon Squeezy
`cancelUserSubscription(subscriptionId)`	Cancels subscription via API

Pricing Logic:

```
base = ₹349/month

months=1  → discount=0%  → total = 349
months=3  → discount=10% → total = 349 * 3 * 0.90 = 942
months=6  → discount=15% → total = 349 * 6 * 0.85 = 1781
months=12 → discount=20% → total = 349 * 12 * 0.80 = 3350
```

Custom checkout data passed: `{ user_id, months, discount_applied }`

4.7. Webhook System

File: `server/src/routes/webhook.js`

Security: Every webhook request is verified using HMAC-SHA256 with a timing-safe comparison against the `LEMONSQUEEZY_WEBHOOK_SECRET`.

Idempotency: Each event's `lsEventId` is stored in the Payment's `idempotencyKeys` array. Duplicate events are silently ignored.

Handled Events (9 total):

Event	Action
`order_created`	Upgrade user to PRO. Calculate expiry (months × 30 days). Create Payment record.
`subscription_created`	Link subscription ID to Order. Set status = 'active'.
`subscription_updated`	Reflect Lemon Squeezy status changes on user.
`subscription_payment_success`	Renew: set status = 'active', update `currentPeriodEnd`, keep tier = 'pro'.
`subscription_payment_failed`	Downgrade: set tier = 'free', status = 'payment_failed'.
`subscription_cancelled`	Mark cancelled (grace period). User keeps PRO until `currentPeriodEnd`.
`subscription_resumed`	Reactivate: tier = 'pro', status = 'active', clear `cancelledAt`.
`subscription_expired`	Auto-downgrade: tier = 'free', status = 'expired'.
`order_refunded`	Immediate downgrade: tier = 'free', status = 'refunded'.

```
Lemon Squeezy
     │
     │  POST /api/webhook/lemonsqueezy
     │  X-Signature: HMAC-SHA256(body, secret)
     ▼
┌──────────────────────────┐
│  1. Verify Signature     │──── Invalid → 401
│  2. Parse event_name     │
│  3. Idempotency check    │──── Duplicate → 200 (skip)
│  4. Extract user_id      │
│     from custom_data     │
│  5. Route to handler     │
│  6. Update User doc      │
│  7. Update Payment doc   │
│  8. Return 200           │
└──────────────────────────┘
```

5. Client Architecture

5.1. App Shell & Routing

Files: `client/src/main.jsx`, `client/src/App.jsx`

```
<BrowserRouter>
  <AuthProvider>          ← Provides user state globally
    <App />               ← Route definitions
    <Toaster />           ← Toast notification layer (top-right)
  </AuthProvider>
</BrowserRouter>
```

Routes:

Path	Component	Access	Description
`/`	`<Landing />`	Public (redirects to /dashboard if logged in)	Landing page with anonymous upload
`/dashboard`	`<Dashboard />`	Protected	Main dashboard
`/category/:categoryName`	`<CategoryView />`	Protected	Category gallery
`*`	Redirect to `/`	—	Catch-all

`ProtectedRoute` **Wrapper:**

- Shows loading spinner while auth state is resolving
- Redirects to `/` if not authenticated

`Stardust` background is always rendered. `Navbar` only renders when user is authenticated.

5.2. State Management

File: `client/src/context/AuthContext.jsx`

SnapSense uses React Context for global auth state — no external state management library.

```
AuthContext = {
  user:      User | null,     // Current authenticated user
  loading:   boolean,         // True while fetching /auth/me
  login:     () => void,      // Redirects to /auth/google
  logout:    () => void,      // POST /auth/logout + clear state
  fetchUser: () => Promise    // Re-fetches user from /auth/me
}
```

Auth State Resolution:

1. On mount, `AuthProvider` calls `GET /auth/me`
2. If session cookie is valid → sets `user`
3. If 401 → sets `user = null`
4. `loading` transitions to `false`
5. All child components can now render based on auth state

5.3. Pages

5.3.1. Landing Page

File: `client/src/pages/Landing.jsx`

The public-facing page. Two-column layout:

```
┌──────────────────────────────────────────────────────────────┐
│  NAVBAR:  Logo  ····· ····· ·····
```

- Max 3 uploads per session

Image Reveal Animation: A curtain effect that slides down over the uploaded image during classification.

5.3.2. Dashboard Page

File: `client/src/pages/Dashboard.jsx`

The primary authenticated experience.

```
┌──────────────────────────────────────────────────────────────┐
│  NAVBAR: Logo · ThemeToggle · Avatar · Name · PRO · Logout │
├──────────────────────────────────────────────────────────────┤
│  SUBSCRIPTION BANNER (PRO users only)                        │
│  "PRO subscription active until Mar 7, 2027"  [▓▓▓▓▓▓▓▓▒▒]  │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  HEADER                                                      │
│  "Welcome, Arihant"  [📷 42 screenshots]  [⚡ Upgrade]      │
│                                                              │
│  UPLOAD ZONE (multi-file, up to 3)                           │
│  ┌──────────────────────────────────────────────────────┐   │
│  │  📊 3/10 used │ Drag or click to upload (max 3)      │   │
│  └──────────────────────────────────────────────────────┘   │
│                                                              │
│  TOP CATEGORIES                                              │
│  ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐             │
│  │📍 12 │ │🗂 8  │ │📚 6  │ │💬 5  │ │✉ 4   │ [Show All] │
│  │Locat.│ │Pay.  │ │Study │ │Quote │ │Mail  │             │
│  └──────┘ └──────┘ └──────┘ └──────┘ └──────┘             │
│                                                              │
│  RECENT SCREENSHOTS (6 per page)                             │
│  ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐ ┌──────┐   │
│  │Thumb │ │Thumb │ │Thumb │ │Thumb │ │Thumb │ │Thumb │   │
│  │Cat.  │ │Cat.  │ │Cat.  │ │Cat.  │ │Cat.  │ │Cat.  │   │
│  │Summ. │ │Summ. │ │Summ. │ │Summ. │ │Summ. │ │Summ. │   │
│  └──────┘ └──────┘ └──────┘ └──────┘ └──────┘ └──────┘   │
│                                                              │
│  UPGRADE MODAL (conditional)                                 │
└──────────────────────────────────────────────────────────────┘
```

State:

- `stats` — `{ total, byCategory: [{ category, count }] }`
- `recent` — paginated screenshot list
- `loading` — initial data fetch
- `billingStatus` — `{ tier, screenshotCount, limit }`
- `showUpgradeModal`, `upgradeTrigger`

Data Loading (3 parallel requests on mount):

1. `GET /api/screenshots/stats` → category counts
2. `GET /api/screenshots?page=1&limit=6` → recent screenshots
3. `GET /api/billing/status` → tier & usage info

Auto-Merge: On mount, checks `localStorage` for `snap_session_id`. If found, calls `POST /auth/merge` to import anonymous session Actions.

Upgrade Detection: Checks for `?upgraded=true` query param → polls `/api/billing/status` until `tier=pro` → shows success toast.

Category Meta: Each category has an associated icon (emoji) and color for visual consistency across all category cards.

5.3.3. Category View Page

File: `client/src/pages/CategoryView.jsx`

Gallery view for a single category.

```
┌──────────────────────────────────────────────────────────────┐
│  ← Back to Dashboard                                         │
│  📍 Location (12 screenshots)                                 │
├──────────────────────────────────────────────────────────────┤
│  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐       │
│  │  Thumbnail   │  │  Thumbnail   │  │  Thumbnail   │       │
│  │  Summary     │  │  Summary     │  │  Summary     │       │
│  │  Date · 98%  │  │  Date · 95%  │  │  Date · 91%  │       │
│  │  [🗺][📁][🗑]│  │  [📁][📅][🗑]│  │  [📁][🗑]    │       │
│  │  2 hours ago │  │  1 day ago   │  │  3 days ago  │       │
│  └──────────────┘  └──────────────┘  └──────────────┘       │
│  ...                                                         │
└──────────────────────────────────────────────────────────────┘
```

State:

- `screenshots` — filtered list for this category
- `loading` — fetch in progress

Action Links per Card:

- 📁 Drive link (if `driveViewLink`)
- 📅 Calendar link (if `calendarEventLink`)
- 🗺 Map link (if `metadata.mapLink`)
- 📊 Sheets (if `sheetsRowNumber`)
- ☑ Tasks (if `taskLink`)
- 🗑 Delete button

Delete: `DELETE /api/screenshots/:id` → refresh list → toast notification.

5.4. Components

5.4.1. Navbar

File: `client/src/components/Navbar.jsx`

```
┌──────────────────────────────────────────────────────────┐
│  🔮 SnapSense    ······ 🎨  [👤 Arihant PRO] [Logout] │
└──────────────────────────────────────────────────────────┘
```

- Fixed position at top
- Logo on left
- Right side: ThemeToggle, user avatar (Google profile picture), first name, PRO badge (if tier='pro'), Sign Out button
- Framer Motion entrance animation (slide down + fade in)

5.4.2. ThemeToggle

File: `client/src/components/ThemeToggle.jsx`

- Toggles `data-theme` attribute on `<html>` element between `'light'` and `'dark'`
- Persists preference to `localStorage('snap_theme')`
- On first load, respects `prefers-color-scheme: dark` system preference
- Renders 🌙 in dark mode, ☀️ in light mode

5.4.3. Stardust (Particle Background)

File: `client/src/components/Stardust.jsx`

- Full-viewport `<canvas>` element positioned behind all content
- 60 particles animated at 60fps via `requestAnimationFrame`
- Each particle has:
  - Random position (x, y)
  - Random radius (1–3px)
  - Random velocity (dx, dy)
  - Sine wave pulsing for opacity variation
  - Viewport edge wrapping (not bouncing)
- Colors adapt to current theme (light vs dark)
- Responsive: recalculates on `window.resize`

5.4.4. UploadZone

File: `client/src/components/UploadZone.jsx`

Props:

Prop	Type	Description
`onSuccess`	`(result) => void`	Callback after successful upload
`screenshotCount`	`number`	Current usage count
`limit`	`number` (default: 10)	Free tier limit
`tier`	`'free' | 'pro'`	User's tier

Features:

- Drag-and-drop zone (react-dropzone)
- Visual drag detection (border color change)
- Multi-file upload (up to 3 files simultaneously)
- File validation: images only, max 10MB per file
- Usage badge: `"📊 3/10 used"` (free) or `"✨ PRO: Unlimited"` (pro)
- **Image reveal loading animation**: A curtain-like overlay slides from top to bottom over the image preview during classification
- Batch progress tracking with progress bar
- Results display with category badge (icon + color)
- Paywall state when limit reached (triggers UpgradeModal)
- Toast notifications: duplicates (orange), errors (red), success (green)

5.4.5. SubscriptionBanner

File: `client/src/components/SubscriptionBanner.jsx`

Fetches subscription data from `GET /api/billing/subscription` on mount.

Display Logic:

Subscription Status	Banner Style	Message
Free tier	Hidden	(not rendered)
Active (>7 days)	Green	"PRO subscription active until {date}"
Active (≤7 days)	Orange/Warning	"Subscription expiring in {N} days"
Cancelled	Orange	"Subscription cancelled — active until {date}"
Payment Failed	Red	"Please update your payment method"
Expired	Red/Critical	"Subscription expired"
Refunded	Red/Critical	"Subscription refunded"

Progress Bar: Visual indicator of time remaining in the billing period (full width = start, empty = expiry).

CTA Button: "Renew Now" shown for warning and critical states.

5.4.6. UpgradeModal

File: `client/src/components/UpgradeModal.jsx`

Props:

Prop	Type	Description
`isOpen`	`boolean`	Controls visibility
`onClose`	`() => void`	Close handler
`trigger`	`'limit_reached' | 'pro_feature' | 'manual'`	Context for header message

Layout:

```
┌──────────────────────────────────────────────────┐
│  ✨ Upgrade to SnapSense PRO                      │
│  "You've reached your free limit..."             │
│                                                   │
│  PLAN SELECTOR                                    │
│  [1 Month ₹349] [3 Months ₹314/mo ★POPULAR]      │
│  [6 Months ₹297/mo] [12 Months ₹279/mo BEST]     │
│                                                   │
│  FEATURE COMPARISON                               │
│  ┌──────────────┬──────────┬──────────┐          │
│  │ Feature      │ Free     │ Pro ☑    │          │
│  │ Uploads      │ 10/month │ Unlimited │          │
│  │ Drive Sync   │ ✗        │ ✓        │          │
│  │ Sheets Log   │ ✗        │ ✓        │          │
│  │ Calendar     │ ✓        │ ✓        │          │
│  │ Tasks        │ ✓        │ ✓        │          │
│  └──────────────┴──────────┴──────────┘          │
│                                                   │
│  Payment: Cards · PayPal · Google · Apple Pay    │
│  🔒 30-day money-back guarantee                   │
│                                                   │
│  [ ⚡ Upgrade for ₹942 ]                          │
└──────────────────────────────────────────────────┘
```

Behavior:

- Plan buttons toggle selected duration
- "Popular" badge on 3-month plan
- "Best Value" badge on 12-month plan

- Discount percentage and savings shown
- Upgrade button triggers `useBilling.handleUpgrade(months)`
- Loading spinner during checkout
- Glassmorphic semi-transparent background overlay

5.4.7. PricingPlans.example.jsx

File: `client/src/components/PricingPlans.example.jsx`

A **reference/example** component (not mounted in the active app). Demonstrates how to build a standalone pricing page that:

- Fetches plans from `GET /api/billing/pricing`
- Displays a 4-card pricing grid
- Shows the current active subscription
- Handles the checkout flow
- Includes an FAQ accordion section

5.4.8. SubscriptionStatus.example.jsx

File: `client/src/components/SubscriptionStatus.example.jsx`

A **reference/example** component (not mounted in the active app). Demonstrates a subscription status widget with:

- Free tier vs PRO tier display
- Color-coded status indicators (green/orange/red)
- Days remaining countdown
- Progress bar for subscription period
- Expiry messaging

5.5. Hooks

5.5.1. `useBilling`

File: `client/src/hooks/useBilling.js`

State: `isLoading`, `error`

Side Effects (on mount):

1. Dynamically loads `https://app.lemonsqueezy.com/js/lemon.js` via script tag injection
2. Sets up `Checkout.Success` event listener → redirects to `/dashboard?upgraded=true`

Function: `handleUpgrade(months, onSuccess)`

1. `POST /api/billing/checkout { months }` → receives `checkoutUrl`

2. Opens Lemon Squeezy checkout overlay (`window.LemonSqueezy.Url.Open(checkoutUrl)`)
3. Polls `GET /api/billing/status` every 3 seconds for up to 30 seconds
4. When `tier === 'pro'` detected → calls `onSuccess` callback
5. On error → sets `error` state

Return Value: `{ isLoading, error, handleUpgrade }`

5.6. Styling & Theming

File: `client/src/index.css`

SnapSense uses a **CSS custom properties (variables) based design system** with light and dark theme support.

Theme Switching Mechanism:

- `<html>` element's `data-theme` attribute controls theme
- CSS variables are defined separately for `[data-theme="light"]` and `[data-theme="dark"]`
- ThemeToggle component toggles this attribute

Color Palette:

Token	Light	Dark
`--bg`	`#fafbfc`	`#0f1117`
`--bg-elevated`	`#ffffff`	`#1a1d27`
`--bg-inset`	`#f3f4f6`	`#242836`
`--border`	`#e5e7eb`	`#2e3345`
`--text`	`#111827`	`#f1f3f8`
`--text-secondary`	`#6b7280`	`#9ca3af`
`--accent`	`#4f46e5` (Indigo)	`#818cf8`
`--accent-soft`	`#eef2ff`	`#1e1b4b`
`--success`	`#059669`	`#34d399`
`--danger`	`#dc2626`	`#f87171`
`--warning`	`#d97706`	`#fbbf24`

Design Tokens:

Token	Value
`border-radius`	12px (default), 8px (sm), 16px (lg)
`navbar-height`	56px

Token	Value
`shadow-xs` → `shadow-lg`	Progressive depth shadows (theme-aware)

Key Styles:

- Loading spinner keyframe animation
- Button variants: primary (accent), secondary (border), ghost (transparent)
- Input focus rings
- Modal overlay with backdrop blur
- Responsive transitions (0.2s ease)

Fonts:

- **Inter** (300–900): Body text, UI elements
- **Space Grotesk** (400–700): Headings, logo

---

6. Core User Flows

6.1. Anonymous Upload Flow

```
User (not logged in)                  Client                          Server
──────────────────────────────────────────────────────────────────────────────
1. Visits /                      Landing page renders
                                 UUID generated → localStorage
2. Drops image on UploadZone
                                 POST /api/screenshots/upload/anonymous
                                 Headers: { x-session-id: UUID }
                                 Body: FormData { screenshot: file }

                                                                  Multer
parses file
                                                                  MD5 hash
generated

AnonymousSession created (if new)
                                                                  Groq AI
classifies image
                                                                  Action
doc saved (24hr TTL)

imageBuffer stored in Action

                                 Receives actionCard:
                                 { actionId, category, metadata }

3. Sees result card              Category badge, summary,
                                 confidence, action links

4. (Repeat up to 3 times)       After 3rd upload:
```

```
                                    limitReached = true
                                    "Sign in to save" CTA
```

6.2. Authenticated Upload Flow

```
User (logged in)                    Client                          Server
──────────────────────────────────────────────────────────────────────────────
1. On Dashboard                  UploadZone visible
   Drops 1–3 images
                                 For each file:
                                 POST /api/screenshots/upload
                                 Body: FormData { screenshot: file }


checkUploadLimit middleware
                                                                    Multer
parses (max 20MB, images)
                                                                    MD5 →
duplicate check
                                                                    Groq AI →
classification
                                                                    Drive
upload → category folder
                                                                    Calendar
event (if applicable)
                                                                    Google
Task (if applicable)
                                                                    Sheets
row (if applicable)

Screenshot doc saved
                                                                    Counts
incremented

                                 Toast: "Screenshot classified as Location"
                                 Dashboard data refreshed
```

Conditional Google Integrations:

Category	Drive	Calendar	Tasks	Sheets Tab
Location	☑	—	—	Locations
Ticket	☑	☑	—	Screenshots
Wallpaper	☑	—	—	—
LinkedIn Profile	☑	—	—	—

Category	Drive	Calendar	Tasks	Sheets Tab
LinkedIn Post	☑	—	—	—
Social Media Post	☑	—	—	—
Payment	☑	☑	—	Transactions
Sensitive Document	☑	—	—	—
Contact	☑	—	—	—
Mail	☑	—	—	—
Quote	☑	—	—	Quotes
WhatsApp Chat	☑	—	—	—
Study Notes	☑	—	☑	—
Other	☑	—	—	—

Additionally, `suggestedAction` from AI can override defaults:

- `suggestedAction = 'calendar'` → creates Calendar event regardless of category
- `suggestedAction = 'task'` → creates Task regardless of category
- `suggestedAction = 'sheet'` → logs to Screenshots tab regardless of category

6.3. Session Merge Flow

```
User signs in after anonymous session
                                    Client                          Server
──────────────────────────────────────────────────────────────────────────────
1. OAuth complete                Redirect to /dashboard
   Dashboard mounts
                                 Reads snap_session_id from localStorage

                                 POST /auth/merge
                                 { sessionId: UUID, verifiedCards: [...] }

                                                                   Find all
pending Actions for sessionId
                                                                   Check
free tier limit (would merge exceed 10?)

                                                                   For each
action (atomically claimed):
                                                                     Upload
imageBuffer to Drive
                                                                     Create
Calendar event (if needed)
                                                                     Create
```

```
Task (if needed)
                                                                                Log to
Sheets (if needed)
                                                                                Create
Screenshot doc

Increment counts

                                                                            Delete
AnonymousSession

                                  Toast: "Saved 3 screenshots from preview
session"
                                  Clears snap_session_id from localStorage
                                  Refreshes dashboard data
```

6.4. Free-to-Pro Upgrade Flow

```
User                                  Client                          Server
──────────────────────────────────────────────────────────────────────────────
1. Hits upload limit              402 LIMIT_REACHED response
   (or clicks Upgrade)
                                  UpgradeModal opens
                                  trigger = 'limit_reached'

2. Selects plan (e.g. 3 months)
   Clicks "Upgrade for ₹942"
                                  POST /api/billing/checkout { months: 3 }
                                                                            Validate
months
                                                                            Check no
active sub
                                                                            Check no
pending payment (<15min)

createCheckoutSession()
                                                                            Return {
checkoutUrl }

                                  window.LemonSqueezy.Url.Open(url)
                                  Lemon Squeezy overlay opens

3. Completes payment
                                                                            POST
/api/webhook/lemonsqueezy
                                                                            event:
order_created
                                                                            →
User.tier = 'pro'
```

```
                                                                    →
subscription.status = 'active'
                                                                    → Payment
record created

                                  Checkout.Success event fires
                                  Redirect to /dashboard?upgraded=true
                                  Polls /api/billing/status
                                  Detects tier = 'pro'

                                  Toast: "Welcome to PRO! 🎉"
                                  UpgradeModal closes
                                  Dashboard refreshes with unlimited badge
```

6.5. Subscription Lifecycle

```
                    ┌──────────────┐
                    │    NONE      │
                    │ (Free tier)  │
                    └──────┬───────┘
                           │ order_created
                           ▼
                    ┌──────────────┐
            ┌──────►│   ACTIVE     │◄──────┐
            │       │ (Pro tier)   │       │
            │       └──┬────┬──────┘       │
            │          │    │              │
            │          │    │              │
            │          │    │ subscription_│
            │          │    │ cancelled    │
            │          │    ▼              │
            │          │  ┌──────────────┐ │
            │          │  │  CANCELLED   │ │ subscription_resumed
            │          │  │ (Pro until   │─┘
            │          │  │  periodEnd)  │
            │          │  └──────┬───────┘
            │          │         │ period expires
            │          │         ▼
            │          │  ┌──────────────┐
            │          │  │   EXPIRED    │
            │          │  │ (Free tier)  │
            │          │  └──────────────┘
            │          │
            │          │ subscription_payment_failed
            │          ▼
            │   ┌──────────────┐
            │   │ PAYMENT      │
            │   │ FAILED       │   subscription_payment_success
            │   │ (Free tier)  │──────────────┘
            │   └──────────────┘
```

```
                │
                │  subscription_payment_success (renewal)
                └──────────────────────────────────

                order_refunded → REFUNDED (Free tier, immediate)
```

6.6. Screenshot Deletion Flow

```
User clicks delete on a screenshot card
                                    Client                          Server
──────────────────────────────────────────────────────────────────────────────
1. Confirm delete (UI)
                                DELETE /api/screenshots/:id

                                                                     Find
screenshot by _id + userId

(ownership verification)

                                                                     if
driveFileId → deleteFileFromDrive()
                                                                     if
calendarEventId → deleteCalendarEvent()
                                                                     if
sheetsRowNumber → deleteSheetRow()


Screenshot.deleteOne()

user.screenshotCount -= 1

user.totalUploads -= 1

user.save()

                                Toast: "Screenshot deleted"
                                Refreshes list / navigates back
```

---

7. AI Classification Engine

System Architecture

```
┌──────────────────────┐
│    Image Upload      │
│  (Buffer + MIME)     │
└──────────┬───────────┘
           │
```

```
              │
              ▼
┌──────────────────────┐
│  Base64 Encoding     │
│  data:image/png;     │
│  base64,iVBOR...     │
└──────────────────────┘
              │
              ▼
┌──────────────────────────────────────────────┐
│                Groq Vision AI                 │
│  Model: meta-llama/llama-4-scout-17b-16e     │
│  Temperature: 0.1                            │
│  Max tokens: 500                             │
│  Messages:                                   │
│    [system] → classification instructions    │
│    [user]   → image_url (base64 data URL)    │
└──────────────────────────────────────────────┘
                        │
                        ▼
┌──────────────────────────────────────────────┐
│             JSON Response Parsing            │
│  {                                           │
│    "category": "Location",                   │
│    "summary": "Eiffel Tower at sunset",      │
│    "date": null,                             │
│    "suggestedAction": "sheet",               │
│    "confidence": 0.97,                       │
│    "locationName": "Eiffel Tower",           │
│    "locationCategory": "Landmark",           │
│    "mapLink": "https://maps.google.com/..." │
│  }                                           │
└──────────────────────────────────────────────┘
                        │
                        ▼ (on parse failure)
                Returns { category: 'Other' }
```

Category-Specific AI Fields

Category	Extra Fields Extracted
Location	`locationName`, `locationCategory`, `mapLink`
Payment	`transactionType` (Credit/Debit), `transactionTime`, `transactionAmount`
Quote	`quoteAuthor`, `quoteGenre`
Ticket	`date` (event date)
Study Notes	`suggestedAction: 'task'`
All	`summary`, `confidence`, `date`, `suggestedAction`

8. Google Integration Layer

OAuth Token Flow

```
┌──────────┐  accessToken + refreshToken  ┌──────────────┐
│   User   │ ──────────────────────────► │  User Model  │
│ Document │                              │  (MongoDB)   │
└──────────┘                              └──────────────┘
                                                 │
                                                 ▼
                                          ┌──────────────┐
                                          │ OAuth2Client │
                                          │ (per-request)│
                                          └──────┬───────┘
                                                 │
                          ┌──────────────────────┼──────────────────────┐
                          │                      │                      │
                          ▼                      ▼                      ▼
                   ┌──────────┐           ┌──────────┐
┌──────────┐
                   │   Drive  │           │  Sheets  │          │ Calendar
│
                   │  API v3  │           │  API v4  │          │ API v3
│
                   └──────────┘           └──────────┘
└──────────┘
                                                                     │
                                                                     ▼
                                                              ┌──────────┐
                                                              │ Tasks    │
                                                              │ API v1   │
                                                              └──────────┘
```

Each service creates its own OAuth2 client from the user's stored tokens. Tokens are refreshed on each Google OAuth re-login.

Google Drive Organization

Each user gets a root folder and 14 category subfolders, lazily created on first upload:

```
User's Google Drive
└── 📷 SnapSense AI (driveRootFolderId)
    ├── 📍 Location
    ├── 🎟️ Ticket
    ├── 🖼️ Wallpaper
    ├── 💼 LinkedIn Profile
    ├── 📝 LinkedIn Post
    ├── 📱 Social Media Post
    ├── 💳 Payment
```

```
    ├── 🔒 Sensitive Document
    ├── 👤 Contact
    ├── 📧 Mail
    ├── 💬 Quote
    ├── 💬 WhatsApp Chat
    ├── 📚 Study Notes
    └── 📂 Other
```

Google Sheets Organization

One spreadsheet per user with 4 tabs:

```
📊 SnapSense AI — Screenshot Log
├── Tab 1: "Screenshots"    (general log for all categories)
├── Tab 2: "Quotes"         (author, genre)
├── Tab 3: "Transactions"   (credit/debit, amount)
└── Tab 4: "Locations"      (name, maps link)
```

---

9. Billing & Payments

Payment Flow Architecture

```
Client                    Server                    Lemon Squeezy
──────────────────────────────────────────────────────────────────

POST /billing/checkout ──►
                          createCheckoutSession() ──►
                          ◄── checkoutUrl ──────────
◄── { checkoutUrl } ─────

LemonSqueezy.Url.Open() ──────────────────────────► Checkout Page
                                                      User Pays
                                                      ◄──────────


                          ◄── POST /webhook ──────── order_created
                          Verify HMAC signature
                          Update User (tier=pro)
                          Create Payment record
                          ──► 200 OK ──────────────►

(Polling /billing/status)
◄── { tier: 'pro' } ─────
Dashboard refreshes
```

Pricing Model

Duration	Monthly Price	Total	Discount	Savings
1 Month	₹349	₹349	—	—
3 Months	₹314	₹942	10%	₹105
6 Months	₹297	₹1,781	15%	₹313
12 Months	₹279	₹3,350	20%	₹838

Anti-Duplicate Measures

1. **Pending payment check:** Won't create a new checkout if a Payment record exists that's < 15 minutes old.
2. **Idempotency keys:** Each webhook event's `lsEventId` is stored. Duplicate events are silently skipped.
3. **Active subscription check:** Can't create a checkout if user already has an active subscription.

---

10. Security Architecture

Authentication & Session Security

- **OAuth 2.0** via Google — no password storage
- **Session cookies:** `httpOnly`, `secure` (production), `sameSite: lax`
- **Session store:** MongoDB (connect-mongo) — server-side session storage
- **Cookie max age:** 7 days

Webhook Security

- **HMAC-SHA256** signature verification with timing-safe comparison
- **Idempotency keys** prevent replay attacks / duplicate processing
- **Raw body parsing** mounted before `express.json()` for accurate signature computation

File Upload Security

- **Multer** validation: images only (MIME type check), max 20MB
- **MD5 hashing** for duplicate detection (prevents re-processing)
- **In-memory** file handling (no temp files on disk)

API Security

- **CORS:** Restricted to `CLIENT_URL` origin with credentials
- **Route guards:** `requireAuth` middleware on all authenticated endpoints
- **Ownership checks:** All screenshot operations verify `userId` matches the authenticated user
- **Tier enforcement:** `checkUploadLimit` and `requirePro` middleware

Data Protection

- **Token storage:** Google OAuth tokens stored in MongoDB (not exposed via API)

- `/auth/me` endpoint excludes sensitive fields (accessToken, refreshToken)
- `rawAI` field excluded from paginated list responses
- **TTL indexes:** Anonymous data auto-deletes after 24 hours

---

11. API Reference

Authentication

Method	Endpoint	Auth	Request	Response
GET	`/auth/google`	No	—	302 Redirect to Google
GET	`/auth/google/callback`	Passport	`?code=`	302 Redirect to /dashboard
POST	`/auth/logout`	Yes	—	`{ success }`
GET	`/auth/me`	Yes	—	`{ success, user }`
POST	`/auth/merge`	Yes	`{ sessionId, verifiedCards }`	`{ merged, results }`

Screenshots

Method	Endpoint	Auth	Request	Response
POST	`/api/screenshots/upload`	Yes	FormData: `screenshot` (file)	`{ success, message, screenshot }` (201)
POST	`/api/screenshots/upload/anonymous`	No	FormData + `x-session-id` header	`{ success, actionCard }`
GET	`/api/screenshots`	Yes	`?page=1&limit=20`	`{ success, data, pagination }`
GET	`/api/screenshots/category/:cat`	Yes	—	`{ success, category, data }`
GET	`/api/screenshots/stats`	Yes	—	`{ success, total, byCategory }`

Method	Endpoint	Auth	Request	Response
DELETE	`/api/screenshots/:id`	Yes	—	`{ success, message }`

Billing

Method	Endpoint	Auth	Request	Response
POST	`/api/billing/checkout`	Yes	`{ months: 1|3|6|12 }`	`{ success, checkoutUrl, months }`
GET	`/api/billing/status`	Yes	—	`{ success, tier, subscription, screenshotCount, limit, countResetAt }`
POST	`/api/billing/verify-upgrade`	Yes	—	`{ success, message }`
GET	`/api/billing/pricing`	No	—	`{ success, currency, plans }`
GET	`/api/billing/history`	Yes	—	`{ success, payments }`
GET	`/api/billing/subscription`	Yes	—	`{ success, hasSubscription, tier, status, ... }`

Webhook

Method	Endpoint	Auth	Request	Response
POST	`/api/webhook/lemonsqueezy`	HMAC-SHA256	Lemon Squeezy event payload	200
POST	`/api/webhook/test-upgrade`	None (dev)	`{ userId }`	`{ success, message }`

Health

Method	Endpoint	Auth	Response
GET	`/health`	No	`{ status: 'OK', dbState }`

12. Free vs Pro Feature Matrix

Feature	Free Tier	Pro Tier

Feature	Free Tier	Pro Tier
AI Screenshot Classification	☑ 14 categories	☑ 14 categories
Monthly Upload Limit	10 screenshots	Unlimited
Anonymous Preview	3 per session	3 per session
Google Drive Sync	☑	☑
Google Sheets Logging	☑	☑
Google Calendar Events	☑	☑
Google Tasks	☑	☑
Category Galleries	☑	☑
Screenshot Deletion (cascade)	☑	☑
Duplicate Detection	☑	☑
Dark Mode	☑	☑
Priority Support	✕	☑
Price	Free	From ₹279/month

13. Data Flow Diagrams

Upload → Classify → Organize (Full Pipeline)

```
┌──────────────────────────┐
│      Image File          │
│   (PNG/JPEG, ≤20MB)      │
└──────────────────────────┘
              │
              ▼
        ┌──────────┐
        │  Multer  │
        │ (Memory) │
        └──────────┘
              │
              ▼
        ┌──────────┐
        │ MD5 Hash │──── Duplicate? → 409
        └──────────┘
              │
              ▼
        ┌──────────────┐
        │  Groq Vision │
        │  AI classify │
        └──────────────┘
              │
              ▼
┌──────────────────────────────────┐
│        Category + Metadata       │
```

```
                        │  { category, summary, confidence,    │
                        │    date, suggestedAction, ... }      │
                        └──────────────────────────────────────┘
                                          │
                                          │
            ┌─────────────────────────────┼─────────────────────────────┐
            │                             │                             │
      ┌─────▼──────┐              ┌──────▼──────┐              ┌──────▼──────┐
      │ Google     │              │ Google      │              │ Google      │
      │ Drive      │              │ Calendar    │              │ Tasks       │
      │            │              │ (conditional)│             │(conditnl)│
      │ 1. Ensure  │              │             │              │            │
      │    folders │              │ Ticket?     │              │ Study      │
      │ 2. Upload  │              │ Payment?    │              │ Notes?     │
      │    to cat. │              │ calendar?   │              │ task?      │
      │    folder  │              │             │              │            │
      │ 3. Set     │              │ → Create    │              │ → Create   │
      │    perms   │              │   event     │              │   task     │
      └────────────┘              └─────────────┘              └────────────┘
            │                             │                             │
            │                             │                             │
            │ fileId, viewLink            │ eventId, eventLink          │ taskId,
taskLink
            │                             │                             │
            └─────────────────────────────┼─────────────────────────────┘
                                          │
                                          │
                                   ┌──────▼──────────┐
                                   │ Google Sheets   │
                                   │ (conditional)   │
                                   │                 │
                                   │ Payment →       │
                                   │   Transactions  │
                                   │ Quote → Quotes  │
                                   │ Location →      │
                                   │   Locations     │
                                   │ Other →         │
                                   │   Screenshots   │
                                   └─────────────────┘
                                          │
                                          │ rowNumber
                                          │
                                   ┌──────▼──────────┐
                                   │  Screenshot     │
                                   │  Document       │
                                   │  (MongoDB)      │
                                   │                 │
                                   │  All IDs +      │
                                   │  links stored   │
                                   └─────────────────┘
```

---

14. Environment Variables

Variable	Required	Description

Variable	Required	Description
`PORT`	No (default: 5000)	Server port
`MONGODB_URI`	Yes	MongoDB Atlas connection string
`SESSION_SECRET`	Yes	Express session encryption secret
`CLIENT_URL`	Yes	Frontend URL for CORS (e.g., `http://localhost:5173`)
`GOOGLE_CLIENT_ID`	Yes	Google OAuth 2.0 Client ID
`GOOGLE_CLIENT_SECRET`	Yes	Google OAuth 2.0 Client Secret
`GOOGLE_CALLBACK_URL`	Yes	OAuth callback URL (e.g., `/auth/google/callback`)
`GROQ_API_KEY`	Yes	Groq SDK API key
`LEMONSQUEEZY_API_KEY`	Yes	Lemon Squeezy API key
`LEMONSQUEEZY_STORE_ID`	Yes	Lemon Squeezy store identifier
`LEMONSQUEEZY_VARIANT_ID`	Yes	Lemon Squeezy product variant ID
`LEMONSQUEEZY_WEBHOOK_SECRET`	Yes	HMAC secret for webhook signature verification
`NODE_ENV`	No	`'production'` for secure cookies + static serving

This document covers every model, route, service, middleware, component, hook, page, style, and flow in the SnapSense AI codebase.